

Class 7: Machine Learning 1

Reta Ado (PID: A17814740)

2026-04-21

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	8
Principal Component Analysis (PCA)	10
Analysis of UK food data	10
Data Input	10
Tidy the data	12
Exploratory analysis	13
PCA	18
Variable Loadings	23

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

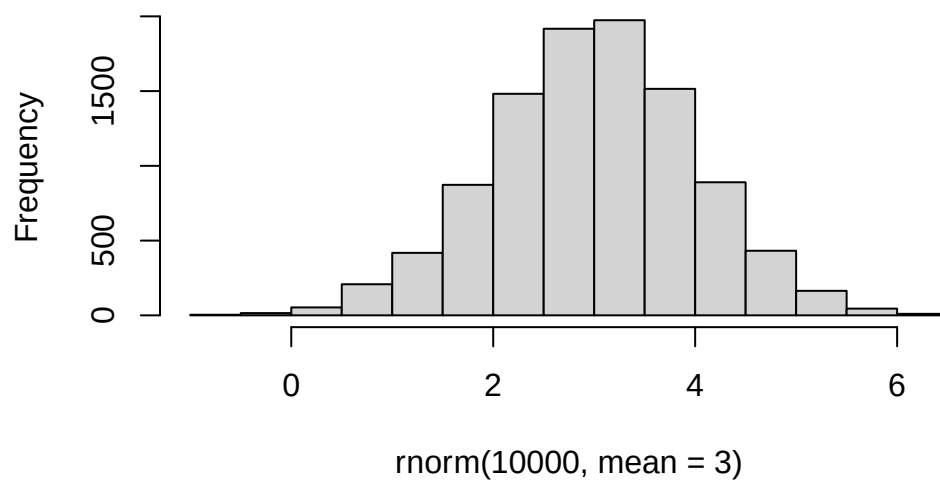
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go to deep let’s makeup some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist(rnorm(10000, mean = 3))
```

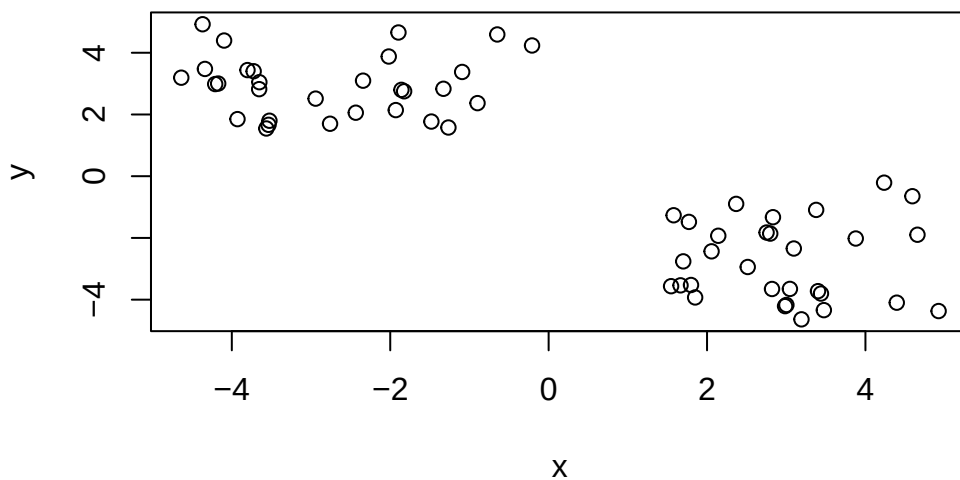
Histogram of rnorm(10000, mean = 3)



```
x <- c(rnorm(30, -3), rnorm(30, +3))
```

```
z <- cbind(x=x, y=rev(x))
```

```
plot(z)
```



```
z <- cbind(x=x, y=rev(x))
z
```

	x	y
[1,]	-4.1723200	3.0021695
[2,]	-3.5639525	1.5462477
[3,]	-3.6523038	3.0463385
[4,]	-1.8245214	2.7517482
[5,]	-2.9420554	2.5138756
[6,]	-1.8586950	2.7980138
[7,]	-3.8021695	3.4382530
[8,]	-4.6381586	3.1921204
[9,]	-0.6483097	4.5926301
[10,]	-2.0185688	3.8778607
[11,]	-4.3396548	3.4751691
[12,]	-1.9298723	2.1419502
[13,]	-2.4348972	2.0573147
[14,]	-2.3429932	3.0956641
[15,]	-1.0916266	3.3776679
[16,]	-1.3281962	2.8323232
[17,]	-2.7597456	1.6996479
[18,]	-3.5251474	1.7977447
[19,]	-1.8962875	4.6567569

[20,]	-4.0975423	4.3955385
[21,]	-3.9284483	1.8497484
[22,]	-4.2100263	2.9852564
[23,]	-4.3691639	4.9238721
[24,]	-3.7242244	3.4010702
[25,]	-1.2653532	1.5783133
[26,]	-0.8977826	2.3677993
[27,]	-3.5363544	1.6653450
[28,]	-3.6547261	2.8205587
[29,]	-1.4803835	1.7709433
[30,]	-0.2093660	4.2357643
[31,]	4.2357643	-0.2093660
[32,]	1.7709433	-1.4803835
[33,]	2.8205587	-3.6547261
[34,]	1.6653450	-3.5363544
[35,]	2.3677993	-0.8977826
[36,]	1.5783133	-1.2653532
[37,]	3.4010702	-3.7242244
[38,]	4.9238721	-4.3691639
[39,]	2.9852564	-4.2100263
[40,]	1.8497484	-3.9284483
[41,]	4.3955385	-4.0975423
[42,]	4.6567569	-1.8962875
[43,]	1.7977447	-3.5251474
[44,]	1.6996479	-2.7597456
[45,]	2.8323232	-1.3281962
[46,]	3.3776679	-1.0916266
[47,]	3.0956641	-2.3429932
[48,]	2.0573147	-2.4348972
[49,]	2.1419502	-1.9298723
[50,]	3.4751691	-4.3396548
[51,]	3.8778607	-2.0185688
[52,]	4.5926301	-0.6483097
[53,]	3.1921204	-4.6381586
[54,]	3.4382530	-3.8021695
[55,]	2.7980138	-1.8586950
[56,]	2.5138756	-2.9420554
[57,]	2.7517482	-1.8245214
[58,]	3.0463385	-3.6523038
[59,]	1.5462477	-3.5639525
[60,]	3.0021695	-4.1723200

```
p <- 1:5
rev(p)
```

```
[1] 5 4 3 2 1
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, centers = 2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

```
      x      y
1  2.929590 -2.738095
2 -2.738095  2.929590
```

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 76.57765 76.57765
(between_SS / total_SS =  86.3 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

\$names

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

\$class

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q: What component of your result object details cluster assignment/membership?

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1  
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

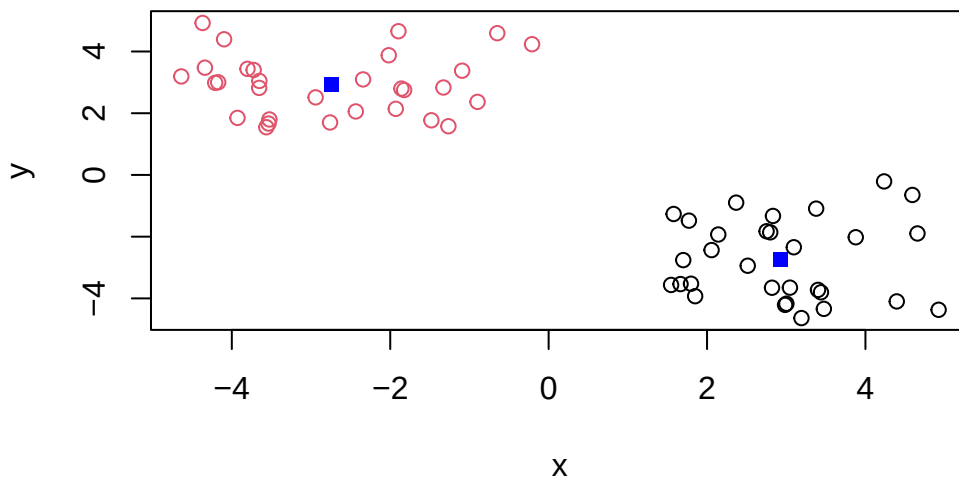
Q. What component of your result object details cluster center?

```
km$centers
```

```
      x      y  
1  2.929590 -2.738095  
2 -2.738095  2.929590
```

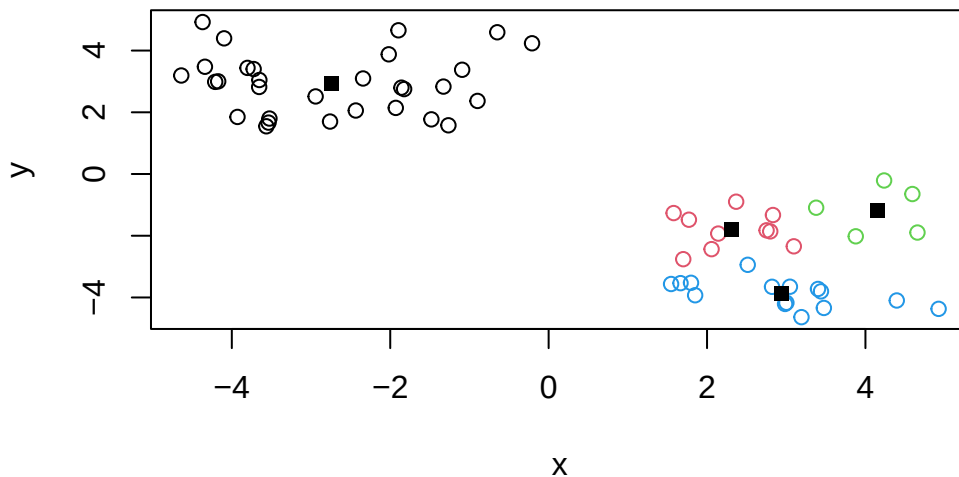
Q. Plot `z` colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col = km$cluster)  
points(km$centers, col = "blue", pch = 15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

```
km4 <- kmeans(z, centers = 4)
plot(z, col = km4$cluster)
points(km4$centers, col = "black", pch = 15)
```



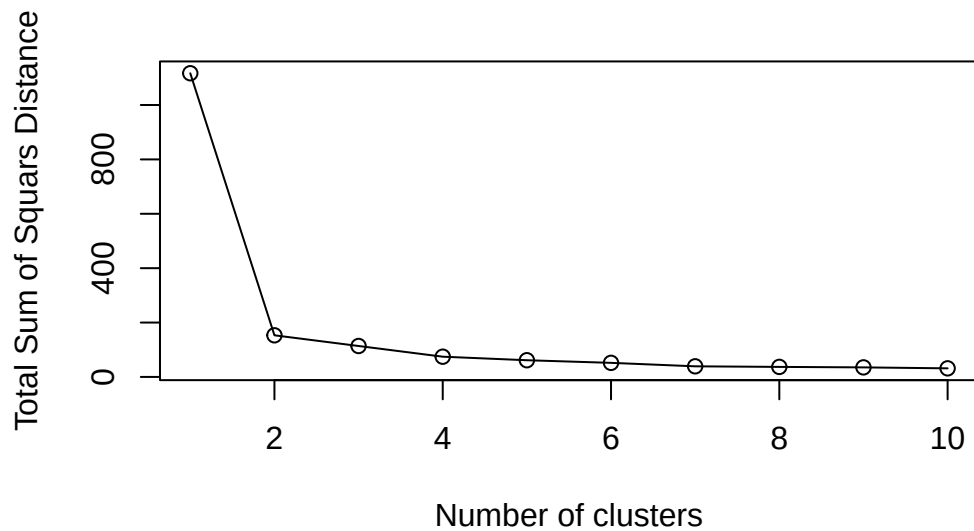
N.B. You need to tell K-means the number of clusters (i.e. set `centers = 2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL

for(i in 1:10) {
  km <- kmeans(z, centers = i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, typ = "o", xlab = "Number of clusters", ylab = "Total Sum of Squares Distance")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

Call:

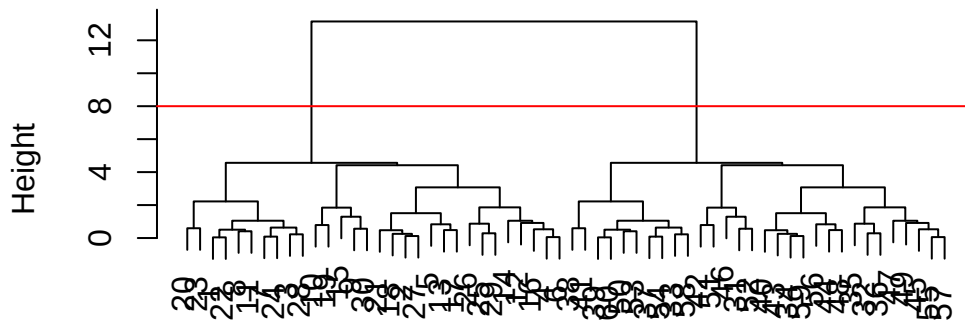
```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.


```
plot(hc)
abline(h=8, col = "red")
```

Cluster Dendrogram



d
hclust (*, "complete")

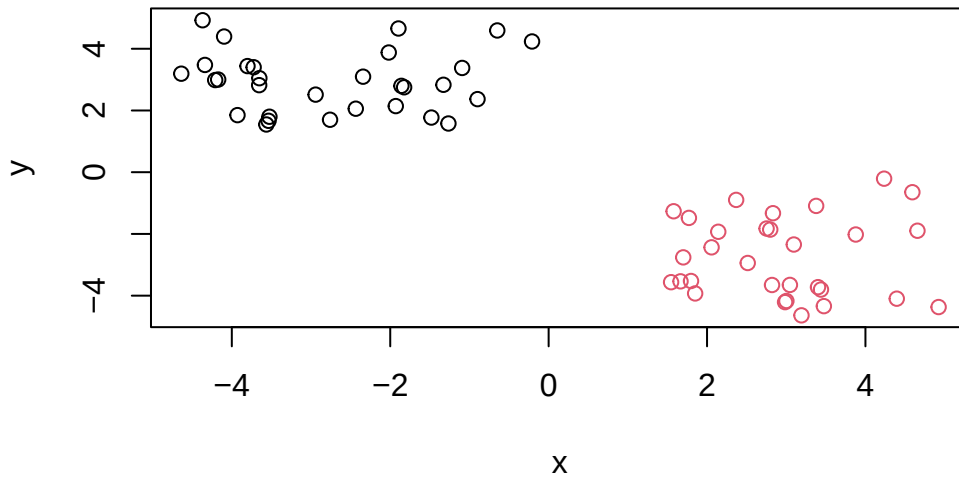
Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, k = 4)
```

```
[1] 1 2 1 2 2 2 1 1 2 2 1 2 2 2 2 2 2 2 1 2 1 1 1 2 2 2 1 2 2 3 3 4 3 3 3 4 4
[39] 4 3 4 3 3 3 3 3 3 3 3 4 3 3 4 4 3 3 3 4 3 4
```

Q. Make a plot of `z` with your `hclust` results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k = 2)
plot(z, col = grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some countries regions being distinct from others.

Data Input

The data is made available in CSV format so we can use the `read.csv(url)`

```
# Store the URL pointing to the UK foods dataset (hosted as a CSV file)
url <- "https://tinyurl.com/UK-foods"
# Read CSV
x <- read.csv(url)
x
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

The data frame x has 17 rows and 5 columns.

```
dim(x)
```

```
[1] 17  5
```

```
ncol(x)
```

```
[1] 5
```

```
nrow(x)
```

```
[1] 17
```

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

The R function that you can use is `dim(x)` since it gives you both dimensions in one call. There are 17 rows and 5 columns. There are technically 4 columns, but the data is not tidied yet to reflect the 4 countries as it considers x.

Tidy the data

Fix anything that went wrong with data import.

Option 1:

```
#Tidy data using option #1
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Option 2:

```
#Tidy data using option #2
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url, row.names = 1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506

Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

```
dim(x)
```

```
[1] 17  4
```

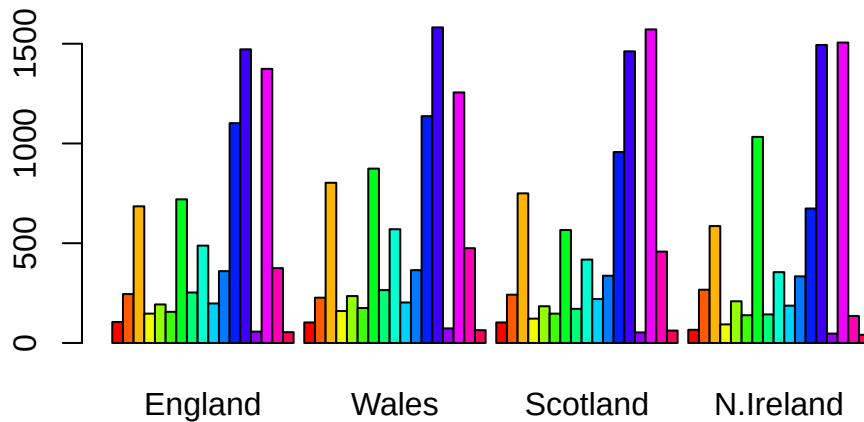
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second approach (`row.names = 1`) to the row-names problem as it is simpler. This is also more robust because it fixes the problem at the import. The option that removes a row is risky because if you run your code twice, it can keep removing rows.

Exporatory analysis

Make some plots to help make sense of obvious trends...

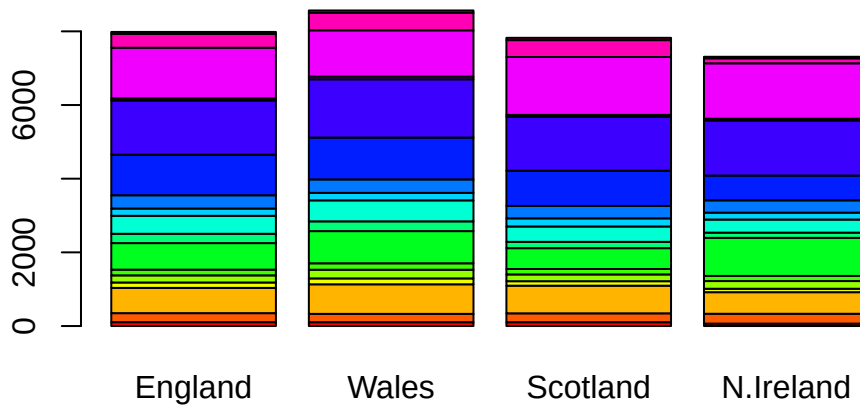
```
#Plot a barplot of the data
barplot(as.matrix(x), beside = TRUE, col = rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

Changing `beside=TRUE` to `beside=FALSE` (or removing it) produces the barplot shown.

```
barplot(as.matrix(x), beside = FALSE, col = rainbow(nrow(x)))
```



```
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
library(tidyr)
```

```
# Convert data to long format for ggplot with `pivot_longer()`  
x_long <- x |>  
  tibble::rownames_to_column("Food") |>
```

```

pivot_longer(cols = -Food,
              names_to = "Country",
              values_to = "Consumption")

dim(x_long)

```

```
[1] 68 3
```

```

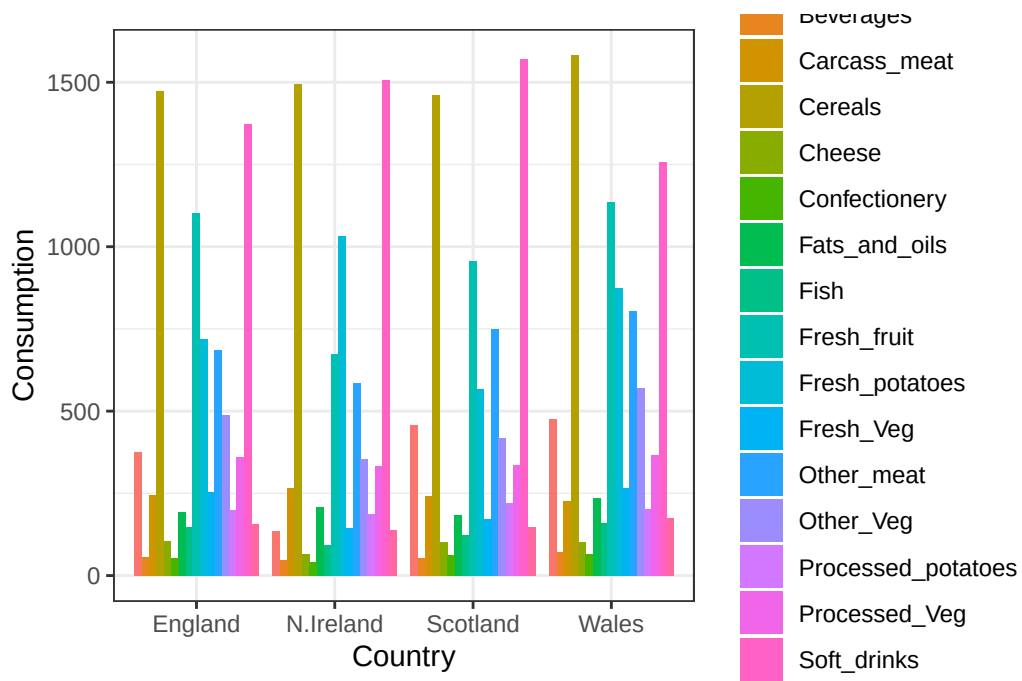
# Create grouped bar plot
library(ggplot2)

```

```

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()

```

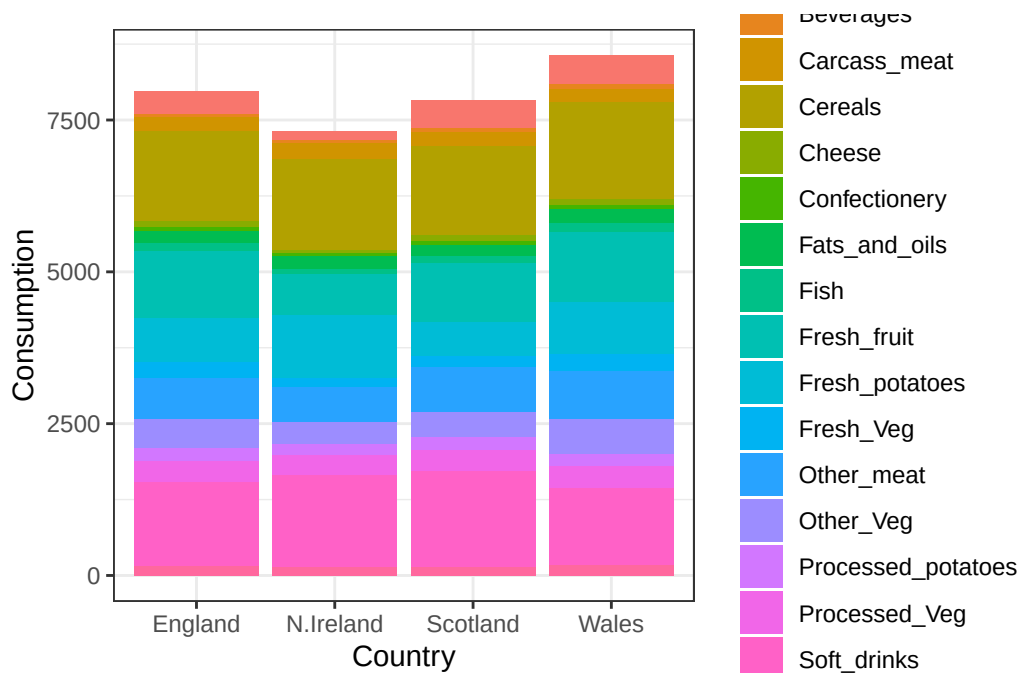


Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Changing `position = "dodge"` to `position = "stack"` in `geom_col()` produces the stacked barplot.

```
# Create stacked bar plot
library(ggplot2)

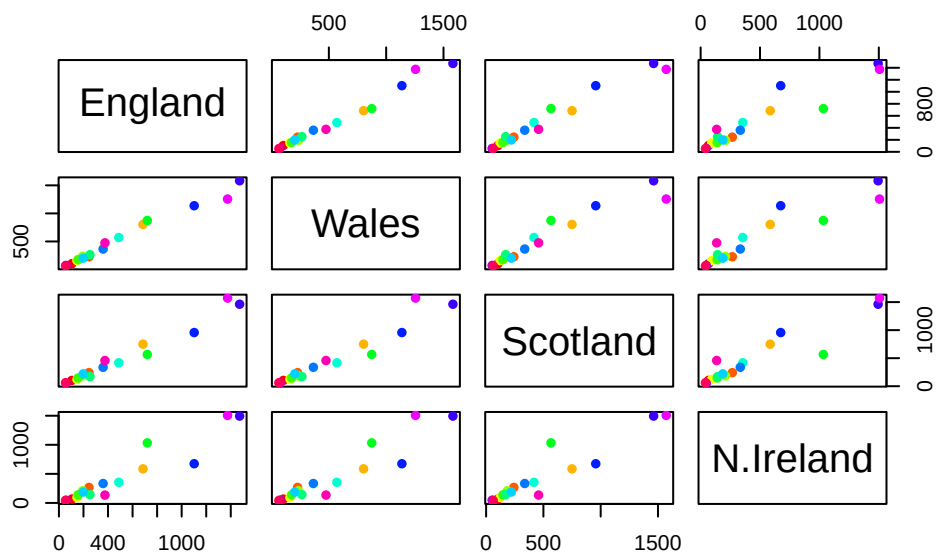
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") + # "stack" accumulates bars vertically
  theme_bw()
```



Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

The `pairs()` function creates a matrix of scatter plots comparing food consumption between every pair of countries, with each colored dot representing one food category. When a point lies on the diagonal, it means both countries consume equal amounts of that food. Points above the diagonal mean the y-axis country consumes more, and points below mean the x-axis country consumes more. Overall, England, Wales, and Scotland show similar consumption patterns, while Northern Ireland deviates more.

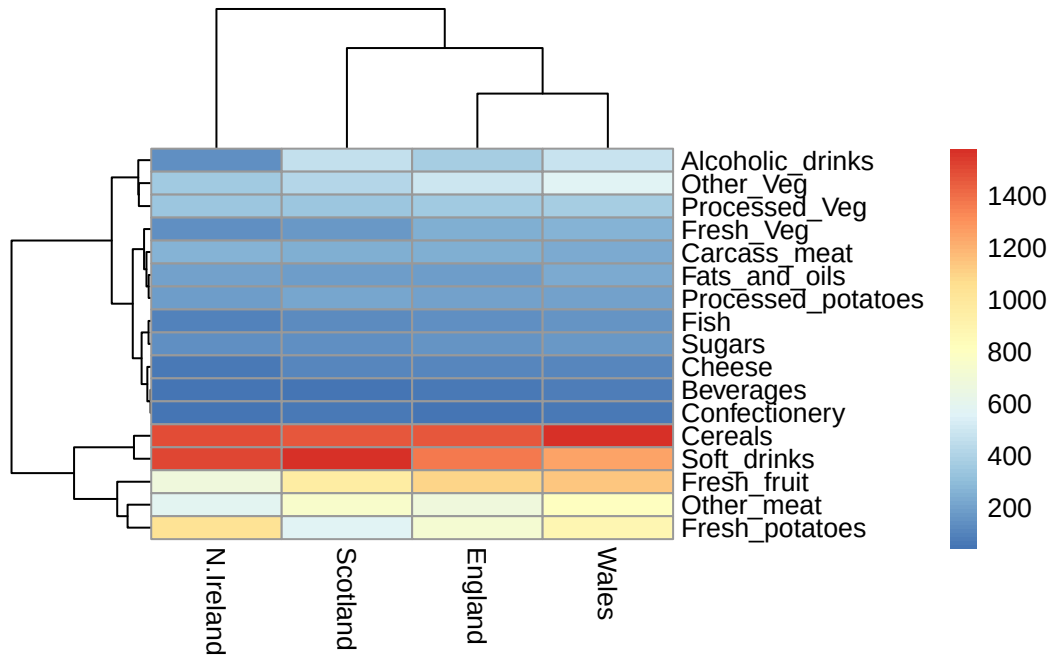
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```

```
# Install pheatmap

library(pheatmap)

# Make a heatmap
pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the figures, England and Wales cluster most closely together, indicating very similar food consumption patterns, while Scotland sits nearby and Northern Ireland is the outlier. N. Ireland shows higher consumption of Fresh potatoes and Other meat, and lower consumption of Fresh fruit compared to the other three countries. However, the heatmap alone makes it somewhat difficult to identify all differences clearly, since the color scale is dominated by high-consumption foods like Fresh potatoes, which can obscure subtler variations in other categories.

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object.

```
# Use the prcomp() PCA function
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main result figures:

```
# Find attributes
attributes(pca)
```

```
$names
[1] "sdev"      "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

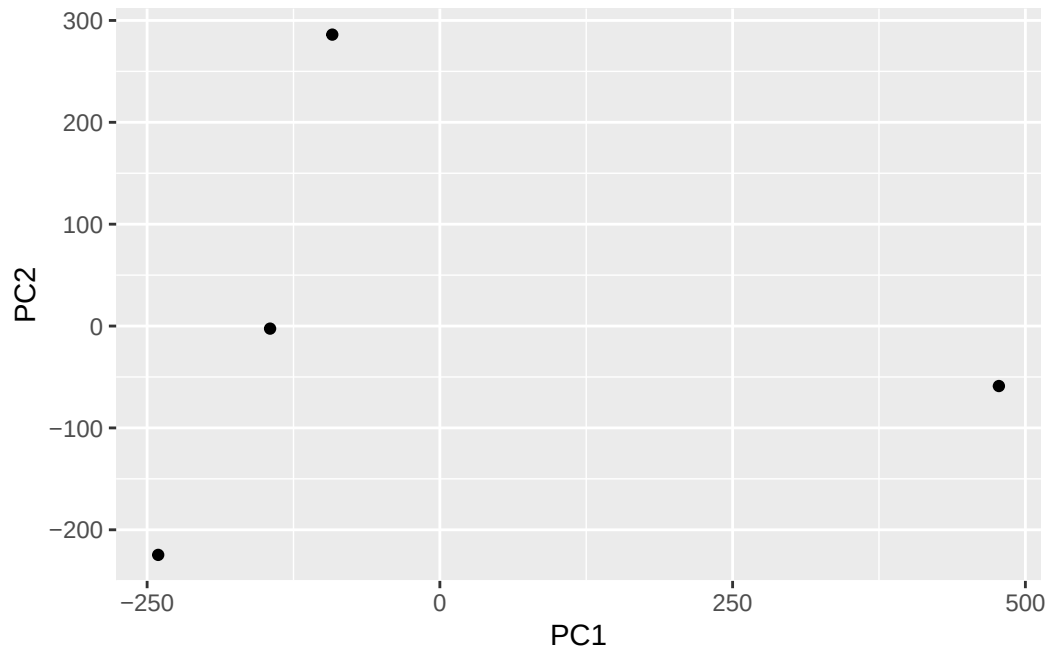
The main result figure from this analysis is called a “PC score plot” or PC1 vs PC2 plot.”

This plot shows how samples (in this case countries) relates to each other along our new PC axis.

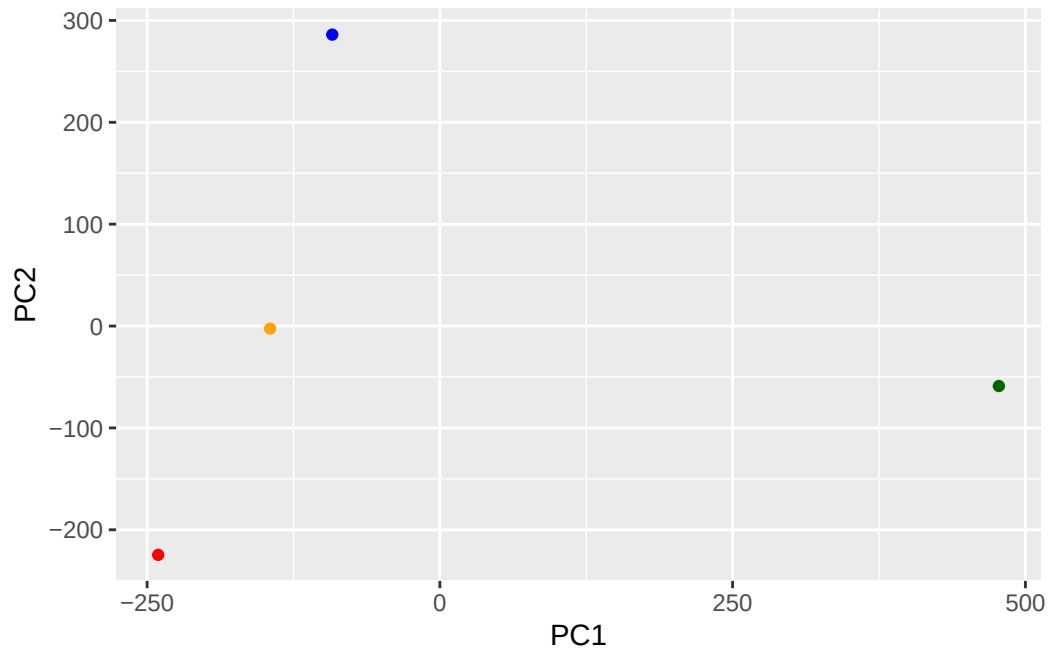
This is our new “reduced-dimensional space.” In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimension data set.

```
library(ggplot2)

ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point()
```



```
mycols <- c("orange", "red", "blue", "darkgreen")  
  
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point(col=mycols)
```

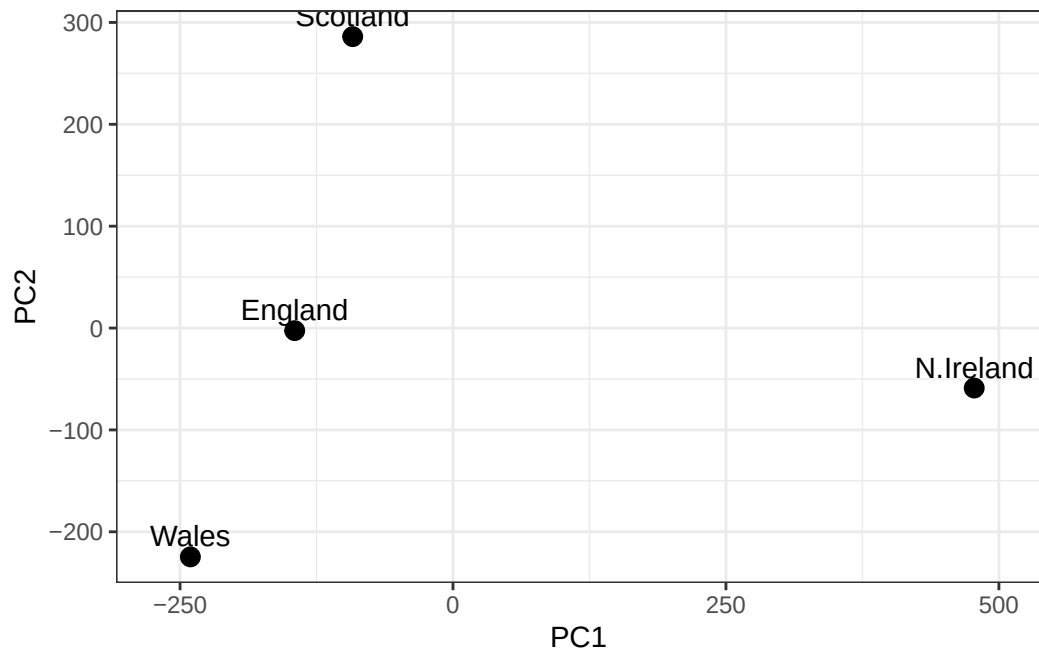


Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Answering Q7

# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```

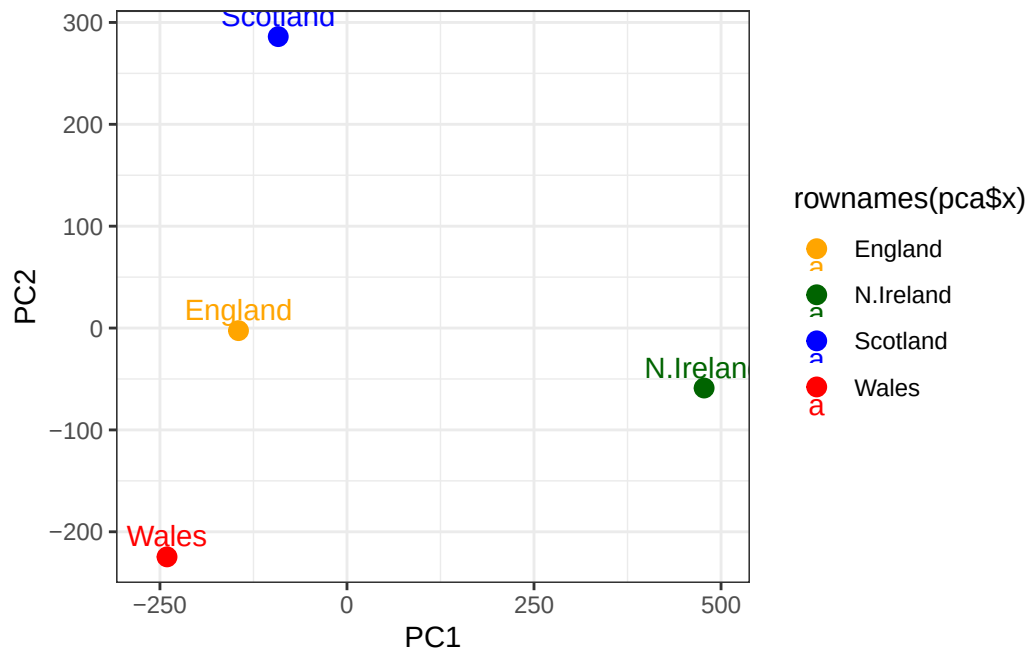


Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
# Answering Q8

# Plot PC1 vs PC2, mapping country names to both label and color aesthetics
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x), color = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  scale_color_manual(values = c("England" = "orange",
                                "Wales" = "red",
                                "Scotland" = "blue",
                                "N.Ireland" = "darkgreen")) +

  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



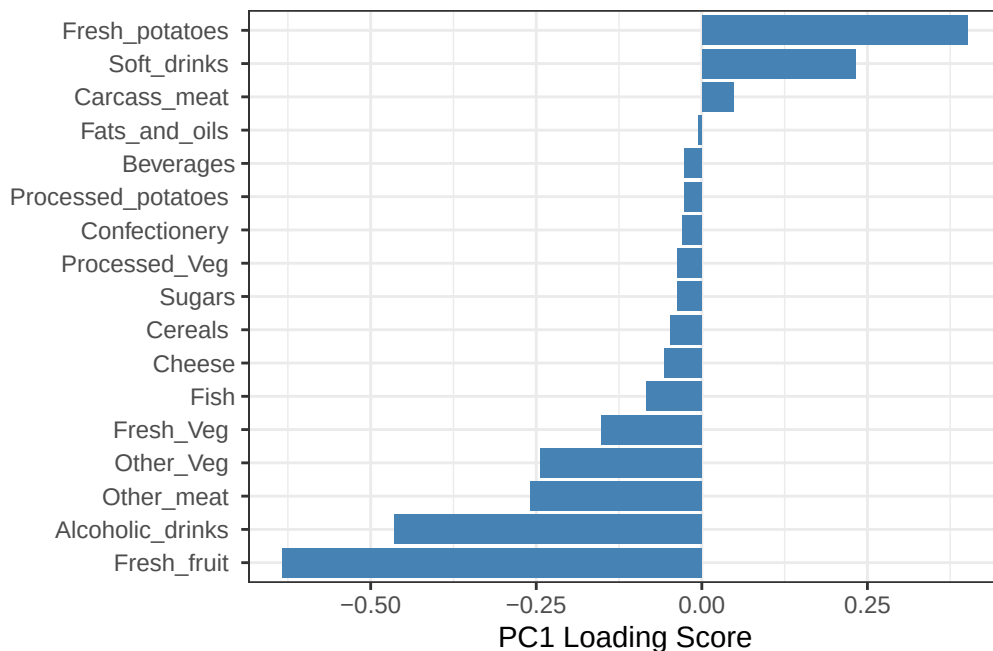
Variable Loadings

```
pca$rotation
```

	PC1	PC2	PC3	PC4
Cheese	-0.056955380	0.016012850	0.02394295	-0.409382587
Carcass_meat	0.047927628	0.013915823	0.06367111	0.729481922
Other_meat	-0.258916658	-0.015331138	-0.55384854	0.331001134
Fish	-0.084414983	-0.050754947	0.03906481	0.022375878
Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.034512161
Sugars	-0.037620983	-0.043021699	-0.03605745	0.024943337
Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	0.021396007
Fresh_Veg	-0.151849942	-0.144900268	0.21382237	0.001606882
Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.031153231
Processed_potatoes	-0.026886233	0.042850761	-0.07364902	-0.017379680
Processed_Veg	-0.036488269	-0.045451802	0.05289191	0.021250980
Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.227657348
Cereals	-0.047702858	-0.212599678	-0.35884921	0.100043319
Beverages	-0.026187756	-0.030560542	-0.04135860	-0.018382072
Soft_drinks	0.232244140	0.555124311	-0.16942648	0.222319484
Alcoholic_drinks	-0.463968168	0.113536523	-0.49858320	-0.273126013

Confectionery -0.029650201 0.005949921 -0.05232164 0.001890737

```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

The two food groups that feature most prominently in PC2 are Soft_drinks (positive loading) and Fresh_potatoes (negative loading). PC2 mainly tells us about the contrast between these two foods, as countries that score high on PC2 consume more soft drinks, while countries that score low on PC2 consume more fresh potatoes. This explains why Scotland sits high on PC2 (higher soft drink consumption) while Wales sits low on PC2 (higher fresh potato consumption).


```
# Generate loadings plot for PC2
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

